

# Árboles de Decisión y Métodos de Ensamble

Guía Maestra de Estudio Handbook Técnico Profesional

|                   |                                                             |
|-------------------|-------------------------------------------------------------|
| <b>Curso</b>      | Análisis Predictivo de Datos ù Machine Learning ù Período 6 |
| <b>Estudiante</b> | Emanuel Cabrera Novoa                                       |
| <b>Docente</b>    | Jhon Quiza                                                  |
| <b>Año</b>        | 2026                                                        |
| <b>Versión</b>    | 2.0 Edición ampliada con guías de ejercicios                |

*Nota: compilar con pdf<sub>l</sub>atex (dos pasadas) o lua<sub>l</sub>atex.*

# Índice general

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Flujo de Decisión ¿Qué Modelo Usar?</b>     | <b>5</b>  |
| 1.1      | Diagrama de Flujo General . . . . .            | 6         |
| 1.2      | Flujo de Regularización de un Árbol . . . . .  | 7         |
| 1.3      | Flujo de Tuning para Boosting . . . . .        | 7         |
| <b>2</b> | <b>Árboles de Decisión Teoría y Práctica</b>   | <b>9</b>  |
| 2.1      | ¿Qué es un Árbol de Decisión? . . . . .        | 9         |
| 2.1.1    | El Algoritmo CART . . . . .                    | 9         |
| 2.2      | Criterios de Impureza . . . . .                | 10        |
| 2.2.1    | Entropía . . . . .                             | 10        |
| 2.2.2    | Ganancia de Información . . . . .              | 10        |
| 2.2.3    | Índice de Gini . . . . .                       | 10        |
| 2.2.4    | Criterios para Regresión . . . . .             | 11        |
| 2.3      | Ventajas y Desventajas . . . . .               | 12        |
| 2.4      | Regularización y Podado . . . . .              | 12        |
| 2.4.1    | Pre-poda (durante la construcción) . . . . .   | 12        |
| 2.4.2    | Post-poda: ccp_alpha . . . . .                 | 12        |
| 2.5      | Invarianza al Escalado . . . . .               | 13        |
| 2.6      | Guía de Ejercicios Árboles . . . . .           | 13        |
| <b>3</b> | <b>Métodos de Ensamble Teoría</b>              | <b>15</b> |
| 3.1      | ¿Qué son los Métodos de Ensamble? . . . . .    | 15        |
| 3.2      | El Trade-off Sesgo-Varianza . . . . .          | 15        |
| 3.3      | Taxonomía de Métodos de Ensamble . . . . .     | 16        |
| <b>4</b> | <b>Bagging y Random Forest</b>                 | <b>17</b> |
| 4.1      | Bootstrap Remuestreo con Reemplazo . . . . .   | 17        |
| 4.2      | Bagging . . . . .                              | 17        |
| 4.2.1    | Out-of-Bag (OOB) Validación Gratuita . . . . . | 17        |
| 4.3      | Random Forest . . . . .                        | 18        |
| 4.4      | Guía de Ejercicios Bagging y RF . . . . .      | 19        |
| <b>5</b> | <b>AdaBoost</b>                                | <b>21</b> |
| 5.1      | Concepto e Historia . . . . .                  | 21        |
| 5.2      | Algoritmo AdaBoost Paso a Paso . . . . .       | 21        |
| 5.3      | Implementación . . . . .                       | 22        |
| 5.4      | Guía de Ejercicios AdaBoost . . . . .          | 22        |
| <b>6</b> | <b>Gradient Boosting</b>                       | <b>25</b> |
| 6.1      | La Idea Central . . . . .                      | 25        |
| 6.2      | Algoritmo Clasificación Binaria . . . . .      | 25        |
| 6.3      | Guía de Ejercicios Gradient Boosting . . . . . | 26        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>7</b>  | <b>Boosting Avanzado: XGBoost, LightGBM y CatBoost</b> | <b>27</b> |
| 7.1       | XGBoost . . . . .                                      | 27        |
| 7.1.1     | Mejoras sobre Gradient Boosting clásico . . . . .      | 27        |
| 7.2       | LightGBM . . . . .                                     | 28        |
| 7.2.1     | Las 4 Innovaciones . . . . .                           | 28        |
| 7.3       | CatBoost . . . . .                                     | 28        |
| 7.3.1     | Innovaciones . . . . .                                 | 28        |
| 7.4       | Comparativa de los 3 Algoritmos Avanzados . . . . .    | 29        |
| <b>8</b>  | <b>Conceptos Fundamentales Los 15 Más Críticos</b>     | <b>31</b> |
| <b>9</b>  | <b>Cheatsheet y Fórmulas de Referencia</b>             | <b>33</b> |
| 9.1       | Tabla de Fórmulas Esenciales . . . . .                 | 33        |
| 9.2       | Guía Rápida de Implementación . . . . .                | 33        |
| <b>10</b> | <b>Guía de Estudio Definitiva</b>                      | <b>37</b> |
| 10.1      | Prerrequisitos . . . . .                               | 37        |
| 10.2      | Roadmap de Aprendizaje 4 Semanas . . . . .             | 37        |
| 10.3      | Errores Comunes y Cómo Evitarlos . . . . .             | 38        |
| 10.4      | Tabla Resumen Final . . . . .                          | 38        |
| <b>11</b> | <b>Conclusiones y Próximos Pasos</b>                   | <b>39</b> |
| 11.1      | Aprendizajes Principales . . . . .                     | 39        |
| 11.2      | Próximos Pasos Recomendados . . . . .                  | 39        |

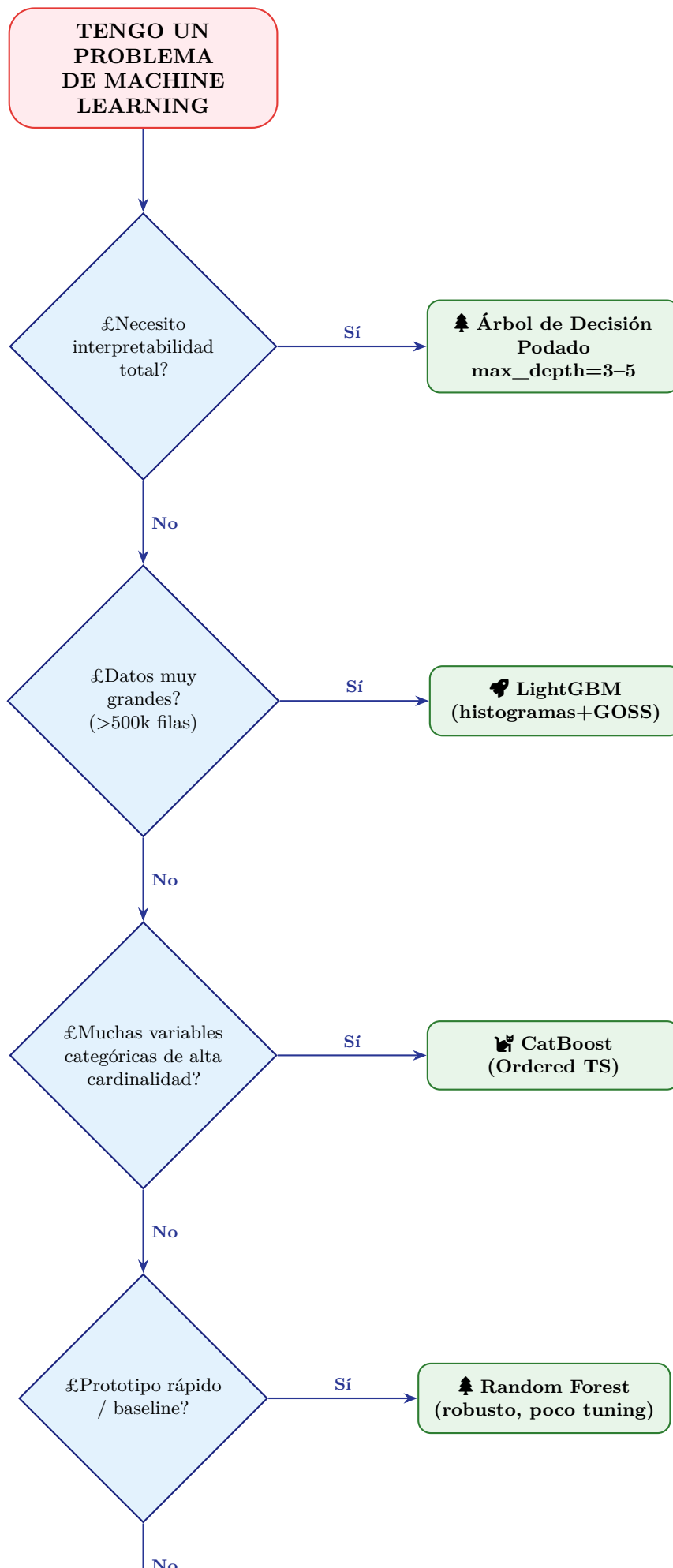


## Capítulo 1

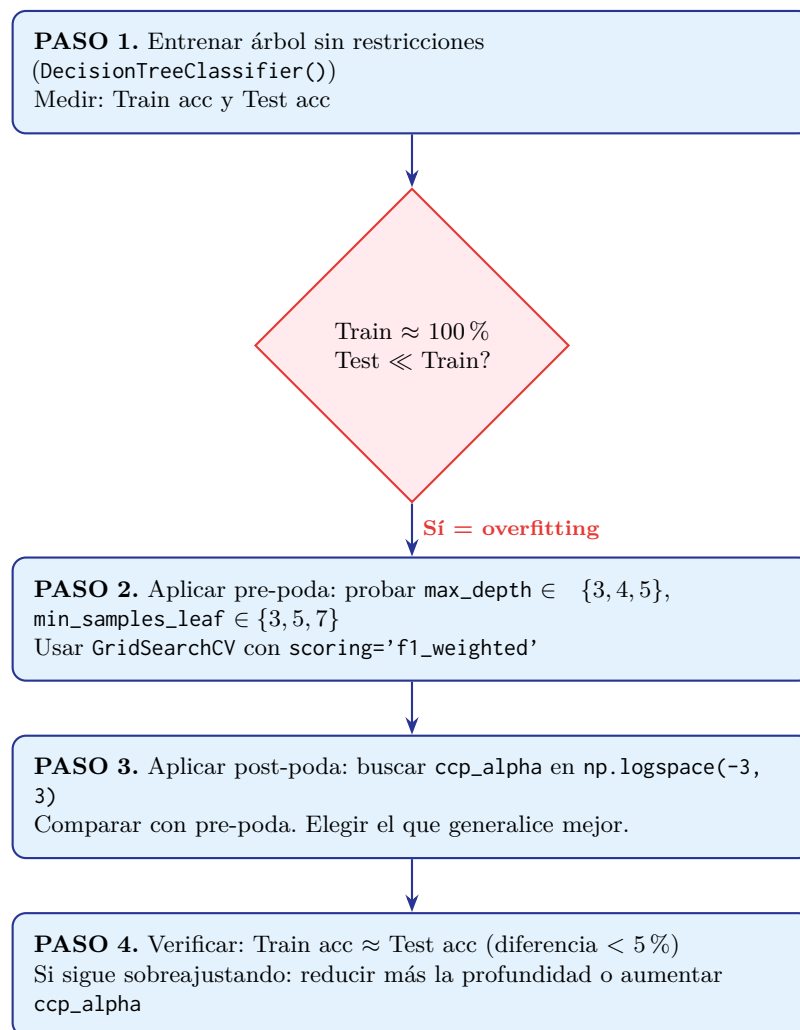
# Flujo de Decisión ¿Qué Modelo Usar?

Esta sección es el mapa de navegación del curso. Antes de escribir una sola línea de código, recorre este flujo para elegir el algoritmo adecuado.

## 1.1 Diagrama de Flujo General



## 1.2 Flujo de Regularización de un Árbol



## 1.3 Flujo de Tuning para Boosting

**Paso 1. Baseline rápido:** entrenar con parámetros por defecto. Obtener F1/accuracy base.

**Paso 2. Número óptimo de árboles:** usar *early stopping* con `learning_rate=0.1`. Observar a qué iteración se estabiliza el error de validación.

**Paso 3. Estructura del árbol:** variar `max_depth` (3–8) y `min_child_weight` o `min_samples_leaf`. Más profundidad = menos sesgo, más varianza.

**Paso 4. Regularización:** submuestreo (`subsample=0.6–0.8`), columnas (`colsample_bytree=0.6–0.8`), L1/L2 (`reg_alpha`, `reg_lambda`).

**Paso 5. Ajuste fino del learning rate:** reducir a 0.05 o 0.01 y aumentar `n_estimators` proporcionalmente. Re-validar con *early stopping*.

**Paso 6. Desbalance de clases:** si recall de clase minoritaria es muy bajo, ajustar `scale_pos_weight` (XGBoost) o el umbral de decisión.

**? ¿CUÁNDO USAR?** Cuándo aplicar cada flujo

| Situación                      | Flujo a seguir                                       |
|--------------------------------|------------------------------------------------------|
| Primera vez con el dataset     | Flujo general (sección 1.1) para elegir familia      |
| Árbol único sobreajusta        | Flujo de regularización (sección 1.2)                |
| Quiero la máxima precisión     | Flujo de tuning boosting (sección 1.3)               |
| Presupuesto de tiempo limitado | Random Forest con RandomizedSearchCV, 30 iteraciones |
| Competencia Kaggle             | Stacking: RF + XGBoost + LightGBM + CatBoost         |



## Capítulo 2

# Árboles de Decisión Teoría y Práctica

### 2.1 ¿Qué es un Árbol de Decisión?

Un árbol de decisión es una estructura jerárquica que divide el espacio de características mediante reglas del tipo *sientonces*. Imita el razonamiento humano: una cadena de preguntas binarias conduce a una decisión final.

#### RESUMEN Elementos del árbol

| Elemento           | Descripción                                     | Analogía                       |
|--------------------|-------------------------------------------------|--------------------------------|
| Nodo raíz (Root)   | Primera pregunta, característica más importante | La pregunta más discriminativa |
| Nodos interiores   | Preguntas intermedias                           | Condiciones anidadas           |
| Ramas (Branches)   | Posibles valores del atributo                   | Respuestas posibles            |
| Hojas (Leaf nodes) | Predicciones finales                            | Resultado: clase o valor       |

#### 2.1.1 El Algoritmo CART

CART (Classification and Regression Trees) construye árboles de forma *greedy* y *top-down*: en cada nodo busca exhaustivamente el atributo y el punto de corte que mejor divida los datos.

Listing 2.1: Pseudocódigo CART

```
1 def CART(nodo, datos, profundidad):
2     if criterio_parada(datos, profundidad):
3         return hoja(clase_mayoritaria(datos))
4
5     mejor_ganancia = -inf
6     for atributo a in características:
7         for valor de corte v in a:
8             izq = datos[a <= v]
9             der = datos[a > v]
10            ganancia = impureza(datos) - impureza_ponderada(izq, der)
11            if ganancia > mejor_ganancia:
12                mejor = (a, v, ganancia)
13
14            nodo.izq = CART(nodo_izq, izq, profundidad+1)
15            nodo.der = CART(nodo_der, der, profundidad+1)
16        return nodo
```

💡 TIP DE ESTUDIO Criterios de parada

El árbol deja de crecer cuando: (1) el nodo es puro (todos de la misma clase), (2) se alcanzó `max_depth`, (3) el nodo tiene menos muestras que `min_samples_split`, o (4) la ganancia de información es cero.

## 2.2 Criterios de Impureza

### 2.2.1 Entropía

Proviene de la teoría de la información de Shannon (1948). Mide la incertidumbre de un conjunto.

📌 FÓRMULA Entropía de Shannon

$$H(S) = - \sum_{c \in \mathcal{C}} p(c) \cdot \log_2 p(c)$$

- $S$  = conjunto de datos,  $\mathcal{C}$  = conjunto de clases,  $p(c)$  = proporción de clase  $c$
- $H(S) = 0 \Rightarrow$  nodo completamente puro (certeza máxima)
- $H(S) = 1 \Rightarrow$  máxima impureza para 2 clases (50/50)
- Rango:  $[0, \log_2 |\mathcal{C}|]$

### Ejemplo numérico Dataset de Tenis

Datos: 4 días se jugó tenis (Yes), 3 no se jugó (No).

$$p(\text{Yes}) = \frac{4}{7} \approx 0,571, \quad p(\text{No}) = \frac{3}{7} \approx 0,429$$

$$H(S) = -(0,571 \cdot \log_2 0,571) - (0,429 \cdot \log_2 0,429) = 0,461 + 0,524 = \mathbf{0,985}$$

*Interpretación:* casi máxima impureza; los datos están muy mezclados.

### 2.2.2 Ganancia de Información

📌 FÓRMULA Ganancia de Información

$$GI(S, a) = H(S) - \sum_{v \in \text{values}(a)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

El árbol selecciona el atributo con **mayor** ganancia (equivale al de **menor** entropía ponderada tras la división).

### 2.2.3 Índice de Gini

Probabilidad de clasificar incorrectamente una muestra aleatoria etiquetada según la distribución del nodo. Más eficiente computacionalmente que la entropía (sin logaritmos).

📌 FÓRMULA Impureza de Gini

$$\text{Gini}(S) = 1 - \sum_i p_i^2$$

$$\text{Gini\_split} = \frac{|L|}{|L| + |R|} \cdot \text{Gini}(L) + \frac{|R|}{|L| + |R|} \cdot \text{Gini}(R)$$

**Ejemplo: nodo con 4 elementos (3A, 1B)**

$$p_A = 0,75, \quad p_B = 0,25$$

$$\text{Gini} = 1 - (0,75^2 + 0,25^2) = 1 - (0,5625 + 0,0625) = \mathbf{0,375}$$

*Interpretación:* 37.5 % de probabilidad de clasificar incorrectamente.

#### RESUMEN Entropía vs. Gini

| Criterio           | Entropía                    | Gini                             |
|--------------------|-----------------------------|----------------------------------|
| Cálculo            | Requiere logaritmos (lento) | Solo potencias (rápido)          |
| Sensibilidad       | Favorece nodos muy puros    | Favorece divisiones equilibradas |
| Resultado práctico | Casi idéntico               | Idéntico en práctica             |
| Default sklearn    | No                          | <b>Sí</b>                        |

#### ? ¿CUÁNDO USAR? Entropía vs. Gini

Usa **entropía** cuando quieras el árbol más informativo posible (diferencias muy pequeñas). Usa **Gini** cuando priorices la velocidad computacional (datasets grandes). En la práctica: no importa cuál elijas, el resultado será prácticamente idéntico.

### 2.2.4 Criterios para Regresión

| Criterio      | Fórmula                                | Cuándo usarlo                         |
|---------------|----------------------------------------|---------------------------------------|
| MSE (default) | $\frac{1}{n} \sum (y_i - \hat{y})^2$   | Errores grandes deben penalizarse más |
| MAE           | $\frac{1}{n} \sum  y_i - \hat{y} $     | Hay outliers que no deben dominar     |
| Friedman MSE  | MSE con corrección                     | Boosting (evalúa potencial de mejora) |
| Poisson       | $-\sum y_i \log \hat{y}_i + \hat{y}_i$ | Variables de conteo (eventos/tiempo)  |

## 2.3 Ventajas y Desventajas

### ✓ Ventajas

- Fácil interpretación (caja blanca)
- Captura relaciones no lineales
- No requiere escalado de variables
- No requiere manejo de nulos
- Selección implícita de características
- Importancia de características disponible

### ✗ Desventajas

- Tendencia al sobreajuste sin regularización
- Alta varianza (sensible a datos)
- Predicciones discontinuas en regresión
- No los más precisos en solitario
- Algoritmo greedy  $\Rightarrow$  no óptimo global

## 2.4 Regularización y Podado

### ⚠ CONCEPTO CRÍTICO

Un árbol **sin regularización siempre sobreajusta**. Es la primera cosa que debes verificar. Train accuracy = 100 % + Test accuracy = 77 % = **señal de alarma**.

### 2.4.1 Pre-poda (durante la construcción)

Listing 2.2: Pre-poda con GridSearchCV

```
1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.model_selection import GridSearchCV
3 import numpy as np
4
5 grid = {
6     'max_depth': [2, 3, 4, 5],           # Profundidad maxima
7     'min_samples_leaf': [3, 5, 7],       # Min muestras por hoja
8     'min_samples_split': [2, 5, 10],     # Min muestras para dividir
9 }
10 gs = GridSearchCV(
11     DecisionTreeClassifier(random_state=1),
12     param_grid=grid,
13     scoring='f1_weighted',
14     cv=5
15 )
16 gs.fit(X_train, y_train)
17 print(f"Mejor: {gs.best_params_}")
```

### 2.4.2 Post-poda: ccp\_alpha

#### ✗ FÓRMULA Cost-Complexity Pruning

El parámetro `ccp_alpha` ( $\alpha$ ) penaliza la complejidad del árbol. A mayor  $\alpha$ , más ramas se podan. El algoritmo elimina primero los “enlaces más débiles” (ramas que menos aportan).

Listing 2.3: Post-poda con `ccp_alpha`

```
1 grid = {'ccp_alpha': np.logspace(-3, 3)} # [0.001 ... 1000]
2 gs = GridSearchCV(
```

```

3 DecisionTreeClassifier(random_state=1),
4 param_grid=grid,
5 scoring='recall'
6 )
7 gs.fit(X_train, y_train)
8 print(f"Mejor alpha: {gs.best_params_}")
9 # Resultado tipico: ccp_alpha=0.00543, recall=0.55

```

### RESUMEN Pre-poda vs Post-poda

|                 | Pre-poda             | Post-poda (ccp_alpha)            |
|-----------------|----------------------|----------------------------------|
| Cuándo          | Durante construcción | Tras construir árbol completo    |
| Velocidad       | Más rápida           | Más lenta                        |
| Calidad         | Puede ser subóptima  | Generalmente más sistemática     |
| Uso recomendado | Datasets grandes     | Cuando importa interpretabilidad |

## 2.5 Invarianza al Escalado

### ⚠ CONCEPTO CRÍTICO

Los árboles **NO se benefician del escalado de variables**. Sus decisiones se basan en umbrales; si escalas, el umbral se ajusta proporcionalmente y el resultado es **idéntico**. Aplicar `StandardScaler` a un árbol no daña, pero es tiempo perdido.

### ? ¿CUÁNDO USAR? Cuándo SÍ escalar con árboles

- Cuando el pipeline incluye también modelos que requieren escalado (SVM, redes neuronales).
- Como paso previo a PCA o reducción de dimensionalidad antes del árbol.
- En algoritmos híbridos donde el árbol es un componente dentro de un pipeline complejo.

## 2.6 Guía de Ejercicios Árboles

### ✎ EJERCICIO 1: Cálculo manual de Entropía y Gini

Dado un nodo con 10 muestras: 6 de clase A y 4 de clase B.

1. Calcula la entropía  $H(S)$ .
2. Calcula el índice de Gini.
3. Tras dividir por un atributo: subconjunto izquierdo: 5A, 1B; derecho: 1A, 3B. Calcula la ganancia de información y el Gini split.
4. ¿Cuál es el mejor criterio aquí? ¿Coinciden entropía y Gini en la decisión?

**Solución parcial:**  $p_A = 0,6$ ,  $p_B = 0,4$ .  $H(S) = -(0,6 \log_2 0,6 + 0,4 \log_2 0,4) = 0,971$ .  
 $Gini(S) = 1 - (0,36 + 0,16) = 0,48$ .

 EJERCICIO 2: Diagnóstico de sobreajuste

Tienes un árbol con Train acc=0.98 y Test acc=0.74. Paso a paso:

1. ¿Qué tipo de error predomina: sesgo o varianza?
2. ¿Qué estrategia de regularización aplicarías primero?
3. Si tras `max_depth=4` obtienes Train=0.84, Test=0.82, ¿está resuelto?
4. ¿Tiene sentido aplicar `StandardScaler` para mejorar el resultado?

## Capítulo 3

# Métodos de Ensamble Teoría

### 3.1 ¿Qué son los Métodos de Ensamble?

Los métodos de ensamble combinan múltiples *clasificadores débiles* (weak learners) para formar un predictor más robusto. Un clasificador débil solo acierta ligeramente mejor que el azar.

#### X<sup>1</sup> FÓRMULA Predictor de Ensamble

$$\hat{f}_{\text{ens}}(x) = \sum_{m=1}^M \alpha_m \cdot \hat{f}_m(x)$$

donde  $\hat{f}_m(x)$  es el  $m$ -ésimo estimador entrenado en  $D_m \subseteq D$ .

### 3.2 El Trade-off Sesgo-Varianza

#### X<sup>1</sup> FÓRMULA Descomposición Sesgo-Varianza-Ruido

$$\mathbb{E}[(y_i - \hat{f}(x_i))^2] = \underbrace{[f(x_i) - \mathbb{E}(\hat{f}(x_i))]^2}_{\text{Sesgo}^2} + \underbrace{\mathbb{E}[(\hat{f}(x_i) - \mathbb{E}(\hat{f}(x_i)))^2]}_{\text{Varianza}} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\text{Ruido irreducible}}$$

#### RESUMEN Componentes del error

| Componente | Definición               | Causa                   | Solución                   |
|------------|--------------------------|-------------------------|----------------------------|
| Sesgo      | Error sistemático        | Modelo demasiado simple | Boosting, modelo complejo  |
| Varianza   | Sensibilidad a los datos | Overfitting             | Bagging, regularización    |
| Ruido      | Variabilidad inherente   | Datos ruidosos          | No reducible con el modelo |

#### 💡 TIP DE ESTUDIO Analogía del arquero

- **Alto sesgo, baja varianza:** flechas juntas pero lejos del centro.
- **Bajo sesgo, alta varianza:** flechas dispersas alrededor del centro.
- **Bajo sesgo, baja varianza:** todas cerca del centro. *El ideal.*
- **Alto sesgo, alta varianza:** el peor caso.

### 3.3 Taxonomía de Métodos de Ensamble

| Característica | BAGGING                     | BOOSTING                              |
|----------------|-----------------------------|---------------------------------------|
| Objetivo       | Reducir <b>varianza</b>     | Reducir <b>sesgo</b>                  |
| Entrenamiento  | Paralelo (independiente)    | Secuencial (corrige al anterior)      |
| Datos          | Bootstrap (con reemplazo)   | Re-pesados o residuos                 |
| Combinación    | Promedio o voto mayoritario | Suma ponderada                        |
| Riesgo         | No reduce sesgo alto        | Puede sobreajustarse                  |
| Paralelizable  | Sí                          | No (secuencial por naturaleza)        |
| Ejemplos       | Bagging, Random Forest      | AdaBoost, GB, XGBoost, LGBM, CatBoost |

#### ? ¿CUÁNDO USAR? Bagging vs Boosting

- Usa **Bagging/RF** cuando el árbol base ya es bastante bueno pero inestable (alta varianza).
- Usa **Boosting** cuando el árbol base es débil (underfitting) y necesitas reducir el sesgo.
- En la práctica, **Boosting casi siempre supera a Bagging** en precisión, pero es más propenso al sobreajuste y requiere más tuning.



## Capítulo 4

# Bagging y Random Forest

### 4.1 Bootstrap Remuestreo con Reemplazo

El bootstrap crea  $M$  conjuntos de datos distintos a partir del dataset original. Selecciona  $n$  muestras **con reemplazo** de un dataset de  $n$  observaciones. Resultado: aproximadamente **63.2 %** de observaciones únicas por muestra (el 36.8 % son duplicados).

#### X<sup>1</sup> FÓRMULA ¿Por qué 63.2%?

La probabilidad de que una muestra específica **no** sea seleccionada en un único sorteo es  $(1 - 1/n)$ . Tras  $n$  sorteos:

$$P(\text{no seleccionado}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0,368$$

Por tanto, el 63.2 % sí aparece en cada muestra bootstrap.

### 4.2 Bagging

#### X<sup>1</sup> FÓRMULA Predictor Bagging

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^M \hat{f}_m(x)$$

$$\text{var}(\hat{f}_{\text{bag}}(x)) = \frac{\sigma^2}{M} \rightarrow 0 \quad \text{cuando } M \rightarrow \infty$$

*La varianza se divide entre el número de estimadores.*

#### 4.2.1 Out-of-Bag (OOB) Validación Gratuita

El ~36.8 % de datos no seleccionados en cada bootstrap sirve como conjunto de validación natural para ese árbol.

#### ⚠ CONCEPTO CRÍTICO

El **error OOB** es un estimador casi tan bueno como la validación cruzada  $k$ -fold, pero computacionalmente mucho más eficiente: no requiere re-entrenar el modelo. Siempre activa `oob_score=True` en `BaggingRegressor` y `RandomForest`.

Listing 4.1: BaggingRegressor con OOB

```

1 from sklearn.ensemble import BaggingRegressor
2 from sklearn.tree import DecisionTreeRegressor
3
4 bag_model = BaggingRegressor(
5     estimator=DecisionTreeRegressor(max_depth=3),
6     n_estimators=50,
7     oob_score=True,      # Validación OOB gratuita
8     n_jobs=-1,           # Paralelizar
9     random_state=1
10 )
11 bag_model.fit(X_train, y_train)
12 print(f'OOB Score (R2): {bag_model.oob_score_:.3f}')
```

### 4.3 Random Forest

Random Forest resuelve el problema de correlación entre árboles en Bagging: si existe una característica muy dominante, todos los árboles la usarán en el nodo raíz y estarán correlacionados entre sí, limitando la reducción de varianza.

**Solución:** en cada nodo se selecciona aleatoriamente un subconjunto de  $m$  características del total de  $p$ , y solo se busca la mejor división dentro de ese subconjunto.

#### X<sup>1</sup> FÓRMULA Algoritmo Random Forest

$$\text{Regresión: } \hat{f}_{\text{rf}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

$$\text{Clasificación: } \hat{f}_{\text{rf}}(x) = \underset{k \in \mathcal{Y}}{\operatorname{argmax}} \sum_{b=1}^B \mathbf{1}[\hat{f}_b(x) = k]$$

$$\text{Valores típicos: } m = \sqrt{p} \text{ (clasificación), } m = p/3 \text{ (regresión)}$$

#### RESUMEN Hiperparámetros clave de Random Forest

| Parámetro        | Descripción               | Efecto                        | Valor típico |
|------------------|---------------------------|-------------------------------|--------------|
| n_estimators     | Número de árboles         | Más → menos varianza          | 100–500      |
| max_features     | Features por nodo ( $m$ ) | Menor → menos correlación     | sqrt / p/3   |
| max_depth        | Profundidad por árbol     | Mayor → menos sesgo           | None         |
| min_samples_leaf | Muestras mínimas/hoja     | Mayor → regularización        | 1            |
| ccp_alpha        | Post-poda por árbol       | Mayor → árboles más simples   | 0            |
| oob_score        | Validación OOB            | Ahorra conjunto de validación | True         |
| n_jobs           | Paralelismo               | -1 = todos los núcleos        | -1           |

#### ⚠ ADVERTENCIA Error común con n\_estimators

Aumentar `n_estimators` indefinidamente no mejora linealmente. Más de 200–500 árboles rara vez mejora el desempeño pero sí aumenta el tiempo de predicción. Usa la curva de aprendizaje para encontrar el punto de rendimientos decrecientes.

Listing 4.2: Random Forest con RandomizedSearchCV

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import RandomizedSearchCV
3 from scipy.stats import loguniform
4
5 dist = {
6     'n_estimators': range(1, 201),
7     'ccp_alpha': loguniform(1e-4, 100),
8     'max_features': [4, 5, 6, 7],
9 }
10 rs = RandomizedSearchCV(
11     RandomForestRegressor(random_state=1),
12     param_distributions=dist,
13     scoring='neg_root_mean_squared_error',
14     n_iter=30, random_state=1
15 )
16 rs.fit(X_train, y_train)
17 print(rs.best_params_)
```

## 4.4 Guía de Ejercicios Bagging y RF

### EJERCICIO 3: Bootstrap manual

Dado el dataset  $\{1, 0, 2, 4, 5, 3, 3, 7, 4, 1\}$  (5 elementos):

1. Genera a mano 3 muestras bootstrap de tamaño 5 (usando una tabla de números aleatorios o semilla fija).
2. ¿Cuántas muestras únicas hay en promedio? ¿Concuerda con el 63.2%?
3. ¿Cuáles serían los elementos OOB de cada muestra?

### EJERCICIO 4: Decorrelación en Random Forest

Explica con tus propias palabras:

1. ¿Por qué Bagging puro NO reduce perfectamente la varianza aunque  $M \rightarrow \infty$ ?
2. ¿Cómo resuelve Random Forest este problema?
3. Si tienes 10 características ( $p = 10$ ), ¿qué valor de  $m$  usarías en clasificación? ¿Y en regresión?
4. Si reduces  $m$  de 10 a 2, ¿qué pasa con el sesgo? ¿Y con la varianza?



## Capítulo 5

# AdaBoost

### 5.1 Concepto e Historia

AdaBoost (Adaptive Boosting) fue propuesto por Freund & Schapire en 1996 y recibió el premio Gödel (equivalente al Nobel en Ciencias de la Computación Teórica). Demostró que podían combinarse clasificadores débiles para formar uno fuerte.

#### RESUMEN Ficha técnica de AdaBoost

|                           |                                               |
|---------------------------|-----------------------------------------------|
| <b>Origen</b>             | Freund & Schapire, 1996 (Premio Gödel)        |
| <b>Idea central</b>       | Re-pesar muestras mal clasificadas            |
| <b>Clasificador base</b>  | Stump (árbol de profundidad 1)                |
| <b>Predicción final</b>   | $\hat{y} = \text{sign}(\sum \alpha_t h_t(x))$ |
| <b>Función de pérdida</b> | Pérdida exponencial                           |
| <b>Limitaciones</b>       | Sensible a ruido; no soporta NaN              |

### 5.2 Algoritmo AdaBoost Paso a Paso

#### **X<sup>1</sup>** FÓRMULA Algoritmo AdaBoost Clasificación Binaria

**Inicialización:**  $w_i^{(1)} = \frac{1}{N}$  para  $i = 1, \dots, N$

**Para**  $t = 1, \dots, T$ :

1. Entrenar  $h_t(x)$  con pesos  $\mathbf{w}^{(t)}$ .
2. Calcular el error ponderado:

$$\varepsilon_t = \frac{\sum_i w_i \cdot \mathbf{1}[y_i \neq h_t(x_i)]}{\sum_i w_i}$$

3. Calcular el peso del estimador:

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

4. Actualizar pesos de muestras:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$$

Mal clasificada:  $w$  **umenta**. Bien clasificada:  $w$  disminuye.

5. Normalizar:  $w_i^{(t+1)} \leftarrow w_i^{(t+1)} / \sum_j w_j^{(t+1)}$

**Predicción final:**  $\hat{y}(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

#### 💡 TIP DE ESTUDIO Intuición de AdaBoost

- El primer clasificador intenta resolver el problema completo con pesos iguales.
- Las muestras fallidas se marcan como “difíciles” y reciben más atención.
- Cada siguiente clasificador se especializa en esas muestras difíciles.
- El consenso ponderado final es más robusto que cualquiera individualmente.

#### ⚠ CONCEPTO CRÍTICO

**AdaBoost no tolera NaN ni variables categóricas directamente.** Siempre aplica `dropna()` y codifica categóricas antes de usarlo.

## 5.3 Implementación

Listing 5.1: AdaBoost con Pipeline completo

```

1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.pipeline import Pipeline
3 from sklearn.model_selection import RandomizedSearchCV
4 from scipy.stats import loguniform
5
6 ada = AdaBoostClassifier(n_estimators=50, random_state=1)
7 model = Pipeline([('preprocessor', preprocessor), ('classifier', ada)])
8 model.fit(X_train, y_train)
9
10 # Sintonización de hiperparametros
11 dist = {
12     'classifier__learning_rate': loguniform(1e-3, 10),
13     'classifier__n_estimators': range(1, 200)
14 }
15 rs = RandomizedSearchCV(
16     model, dist, scoring='f1_weighted',
17     n_iter=20, random_state=1
18 )
19 rs.fit(X_train, y_train)
20 # Best: learning_rate=1.63, n_estimators=62

```

## 5.4 Guía de Ejercicios AdaBoost

### ✍ EJERCICIO 5: Seguimiento manual de AdaBoost

Tienes 6 muestras con etiquetas  $y = [+1, +1, +1, -1, -1, -1]$ . El clasificador  $h_1$  predice  $[+1, +1, -1, -1, -1, +1]$ .

1. ¿Cuáles muestras clasifica mal  $h_1$ ?

2. Calcula  $\varepsilon_1$  con pesos iniciales iguales.
3. Calcula  $\alpha_1$ .
4. Actualiza los pesos de las muestras mal clasificadas vs bien clasificadas.
5. Normaliza los pesos para que sumen 1.





## Capítulo 6

# Gradient Boosting

### 6.1 La Idea Central

Gradient Boosting reformula el boosting como un **problema de optimización**: en lugar de re-pesar muestras, entrena cada nuevo árbol para predecir el **gradiente negativo** de la función de pérdida respecto a las predicciones actuales es decir, los **residuos**.

#### X<sup>1</sup> FÓRMULA Gradient Boosting: actualización

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

$$F_M(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \cdots + \eta \cdot h_M(x)$$

donde  $\eta$  es el *learning rate* y  $h_m$  es el  $m$ -ésimo árbol que predice los residuos.

### 6.2 Algoritmo Clasificación Binaria

#### X<sup>1</sup> FÓRMULA Gradient Boosting: clasificación

1. **Inicialización:**  $F_0(x) = \log\left(\frac{P(y=1)}{P(y=0)}\right)$
2. **Para**  $m = 1, \dots, M$ :
  - a) Residuos (gradiente negativo de la log-loss):  $r_{im} = y_i - \hat{p}_{m-1}(x_i)$  donde  $\hat{p}_{m-1} = \sigma(F_{m-1}(x_i)) = \frac{1}{1+e^{-F_{m-1}(x_i)}}$
  - b) Entrenar  $h_m(x)$  (árbol de regresión) para predecir  $\{r_{im}\}$
  - c) Actualizar:  $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$
3. **Predicción final:**  $\hat{p}(x) = \sigma(F_M(x))$

**Para regresión (MSE):**  $F_0(x) = \bar{y}$ ;  $r_{im} = y_i - F_{m-1}(x_i)$ ;  $\hat{y}(x) = F_M(x)$ .

**RESUMEN** Hiperparámetros clave de Gradient Boosting

| Parámetro        | Descripción           | Impacto                         | Rango típico |
|------------------|-----------------------|---------------------------------|--------------|
| n_estimators     | Número de árboles     | + → - sesgo, riesgo overfitting | 50–500       |
| learning_rate    | Escala por árbol      | - → necesita más árboles        | 0.01–0.3     |
| max_depth        | Profundidad por árbol | + → + varianza                  | 3–6          |
| subsample        | Fracción datos/árbol  | - → más regularización          | 0.5–1.0      |
| n_iter_no_change | Early stopping        | Previene overfitting            | 5–15         |

**TIP DE ESTUDIO** Regla práctica learning rate vs n\_estimators

Empieza con `lr=0.1` y `n_estimators=100`. Si quieres afinar: reduce `lr` a 0.05 o 0.01 y multiplica `n_estimators` proporcionalmente. Valida con early stopping.

### 6.3 Guía de Ejercicios Gradient Boosting

**EJERCICIO 6:** Trazado de curva de aprendizaje

Con `GradientBoostingClassifier` y el dataset Adult:

1. Entrena con `learning_rate=0.1` y `n_estimators` en `{10, 50, 100, 200, 500}`.
2. Registra el F1 de train y test para cada valor.
3. ¿En qué punto empieza el overfitting?
4. Repite con `learning_rate=0.01`. ¿Cambia el punto de overfitting?
5. ¿Qué concluyes sobre la relación entre `lr` y `n_estimators`?

## Capítulo 7

# Boosting Avanzado: XGBoost, LightGBM y CatBoost

### 7.1 XGBoost

#### 7.1.1 Mejoras sobre Gradient Boosting clásico

##### RESUMEN Innovaciones de XGBoost

| Mejora                                  | Problema que resuelve                       | Cómo funciona                                                         |
|-----------------------------------------|---------------------------------------------|-----------------------------------------------------------------------|
| Regularización ( $\gamma$ , $\lambda$ ) | GB puede sobreajustarse                     | $\gamma$ penaliza hojas extra; $\lambda$ es L2 sobre pesos            |
| Newton Boosting                         | GB usa solo 1er orden (lento)               | Usa el Hessiano (2ª derivada); converge más rápido                    |
| Cuantiles ponderados                    | Evaluar todos los splits es $O(n \times p)$ | Agrupar en bins ponderados por el Hessiano; $O(\text{bins} \times p)$ |
| NaN nativo                              | Requería imputación previa                  | Aprende la dirección óptima para valores faltantes                    |
| Categorías nativas                      | One-Hot crea matrices dispersas             | Ordena por estadísticas de gradiente                                  |

Listing 7.1: XGBoost con early stopping y categorías nativas

```
1 from xgboost import XGBClassifier
2 from sklearn.model_selection import train_test_split
3
4 # Convertir categoricas a tipo 'category'
5 X[cat_cols] = X[cat_cols].astype('category')
6
7 X_tr, X_val, y_tr, y_val = train_test_split(
8     X_train, y_train, test_size=0.1, stratify=y_train
9 )
10
11 model = XGBClassifier(
12     grow_policy='lossguide',      # Leaf-wise (como LightGBM)
13     tree_method='hist',          # Histogramas (rapido)
14     enable_categorical=True,      # Soporte nativo categoricas
15     early_stopping_rounds=10,
16     random_state=1
17 )
18 model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
```

## 7.2 LightGBM

### 7.2.1 Las 4 Innovaciones

#### RESUMEN Innovaciones de LightGBM

| Innovación       | Descripción                                                           | Beneficio                                   |
|------------------|-----------------------------------------------------------------------|---------------------------------------------|
| Histogramas      | Discretiza features en bins ( $\approx 255$ ); busca splits en bordes | 40,000× más rápido con 10M filas            |
| Leaf-wise growth | Solo expande la hoja con mayor ganancia                               | Mismo número de nodos, menor error          |
| GOSS             | Usa muestras con gradiente alto; muestra aleatoria del resto          | Reduce datos por árbol sin perder precisión |
| EFB              | Fusiona features que nunca son nonzero simultáneamente                | Reduce dimensionalidad en datos dispersos   |

#### ADVERTENCIA Leaf-wise y overfitting

Leaf-wise puede crear árboles muy desequilibrados y sobreajustar. Controla la complejidad con `num_leaves` (no solo con `max_depth`). Regla:  $\text{num\_leaves} < 2^{\text{max\_depth}}$ .

Listing 7.2: LightGBM con early stopping

```

1 import lightgbm as lgb
2 from lightgbm import LGBMClassifier
3
4 lgb_model = LGBMClassifier(objective='binary', verbose=0)
5 lgb_model.fit(
6     X_tr, y_tr,
7     eval_set=[(X_val, y_val)],
8     eval_metric='logloss',
9     callbacks=[lgb.early_stopping(stopping_rounds=10)],
10 )

```

## 7.3 CatBoost

### 7.3.1 Innovaciones

#### RESUMEN Innovaciones de CatBoost

| Innovación                | Problema                                              | Solución                                                                                  |
|---------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Ordered Target Statistics | Target encoding clásico introduce fuga de información | Calcula la media del target usando <b>solo</b> las muestras anteriores al orden aleatorio |
| Feature combinations      | Interacciones entre categorías                        | Crea combinaciones automáticas                                                            |
| Árboles simétricos        | Predicción costosa con árboles asimétricos            | Misma condición en todos los nodos de un nivel; predicción $O(1)$                         |

**⚠ CONCEPTO CRÍTICO**

**CatBoost soporta NaN en numéricas, pero NO en categóricas.** Aplica `dropna(subset=categorical_features)` antes de entrenar.

Listing 7.3: CatBoost con soporte nativo de categóricas

```

1 from catboost import CatBoostClassifier
2
3 cat_model = CatBoostClassifier(verbose=0)
4 cat_model.fit(
5     X_tr, y_tr,
6     cat_features=categorical_features, # Indicar categóricas
7     eval_set=(X_val, y_val),
8     early_stopping_rounds=10
9 )

```

## 7.4 Comparativa de los 3 Algoritmos Avanzados

| Criterio        | XGBoost                         | LightGBM                      | CatBoost                      |
|-----------------|---------------------------------|-------------------------------|-------------------------------|
| Velocidad       | Media                           | Muy alta                      | Media-alta                    |
| Datos >1M filas | Lento                           | Excelente                     | Medio                         |
| Categorías      | Encoding manual                 | Básico nativo                 | Avanzado nativo (sin leakage) |
| Valores nulos   | Sí (nativo)                     | Sí (nativo)                   | Solo numéricas                |
| Regularización  | $\gamma$ , $\lambda$ , $\alpha$ | <code>reg_alpha/lambda</code> | <code>l2_leaf_reg</code>      |
| Crecimiento     | Level-wise (configurable)       | Leaf-wise (default)           | Simétrico                     |
| Mejor para      | Datos mixtos, tuning fino       | Datasets muy grandes          | Muchas categóricas            |



## Capítulo 8

# Conceptos Fundamentales Los 15 Más Críticos

### 1. Sobreajuste (Overfitting)

Train acc=100 %, test acc=77 % = señal de alarma. El árbol memorizó el ruido. *Siempre regularizar.*

### 2. Entropía y Ganancia de Información

La entropía mide cuánta impureza hay; la ganancia mide cuánto la reduce una división. *Corazón del árbol: sin esto, no sabe qué pregunta hacer primero.*

### 3. Impureza de Gini

Probabilidad de clasificar mal una muestra aleatoria. Más eficiente que entropía. *En práctica, entropía y Gini dan resultados casi idénticos.*

### 4. Podado (Pruning)

Pre-poda: más eficiente. Post-poda con `ccp_alpha`: más sistemática. *Siempre preferir post-poda cuando el tiempo lo permite.*

### 5. Feature Importance

Importancia relativa basada en reducción de impureza. Herramienta de selección. *No indica causalidad. Solo relevancia predictiva.*

### 6. Bootstrap

≈63.2% de muestras únicas por muestra bootstrap. Permite crear  $M$  datasets distintos. *Fundamento de todo Bagging y Random Forest.*

### 7. Out-of-Bag (OOB) Error

El 36.8 % no seleccionado sirve como validación gratuita por árbol.  $OOB \approx CV\ k\text{-fold}$ , pero mucho más eficiente computacionalmente.

### 8. Decorrelación (Random Forest)

RF selecciona  $m < p$  features por nodo, reduciendo correlación entre árboles. *Sin decorrelación, el promedio de árboles similares mejora poco la varianza.*

### 9. Sesgo-Varianza Trade-off

Error = Sesgo<sup>2</sup> + Varianza + Ruido. Bagging → reduce varianza. Boosting → reduce sesgo. *Este tradeoff determina qué familia de algoritmo usar.*

### 10. Learning Rate ( $\eta$ )

Escala la contribución de cada árbol. Menor  $\eta \Rightarrow$  más árboles, mejor generalización. *Regla:  $lr=0.1$ , 100 árboles como punto de partida.*

**11. Early Stopping**

Detener cuando el error de validación deja de mejorar. Previene overfitting automáticamente. *Disponible en XGBoost, LightGBM y CatBoost. Siempre activarlo.*

**12. Target Encoding**

Reemplaza cada categoría con la media del target. Útil para alta cardinalidad. *Siempre usar TargetEncoder de sklearn (cross-fitting interno) para evitar leakage.*

**13. Residuos en Gradient Boosting**

Los residuos = gradiente negativo de la pérdida. Cada árbol corrige los errores actuales. *Clave: GB predice el ERROR, no y directamente.*

**14. Hessiano (Newton Boosting XGBoost)**

La segunda derivada de la pérdida permite una aproximación cuadrática más precisa. *XGBoost converge con menos árboles que GB clásico.*

**15. Stacking vs Averaging**

Averaging: promedio simple. Stacking: meta-modelo que aprende cómo combinar modelos base. *En competencias, stacking multi-nivel da típicamente el mejor resultado.*



## Capítulo 9

# Cheatsheet y Fórmulas de Referencia

### 9.1 Tabla de Fórmulas Esenciales

| Concepto          | Fórmula                                                                        | Rango / Interpretación            |
|-------------------|--------------------------------------------------------------------------------|-----------------------------------|
| Entropía          | $H(S) = -\sum_c p(c) \log_2 p(c)$                                              | $[0, \log_2 k]$ ; 0=puro          |
| Ganancia info.    | $GI(S, a) = H(S) - \sum_v \frac{ S_v }{ S } H(S_v)$                            | $[0, H(S)]$ ; mayor=mejor split   |
| Gini              | $\text{Gini}(S) = 1 - \sum_i p_i^2$                                            | $[0, 1 - 1/k]$ ; 0=puro           |
| Gini split        | $\frac{ L }{ L + R } \text{Gini}(L) + \frac{ R }{ L + R } \text{Gini}(R)$      | Minimizar                         |
| Bagging           | $\hat{f}_{\text{bag}}(x) = \frac{1}{M} \sum \hat{f}_m(x)$                      | Promedio $M$ modelos              |
| Var. bagging      | $\text{var}(\hat{f}_{\text{bag}}) = \sigma^2/M \rightarrow 0$                  | Decrece con $M$                   |
| RF regresión      | $\hat{f}_{\text{rf}}(x) = \frac{1}{B} \sum_b \hat{f}_b(x)$                     | Promedio de $B$ árboles           |
| RF clasificación  | $\hat{f}_{\text{rf}}(x) = \text{argmax}_k \sum_b \mathbf{1}[\hat{f}_b(x) = k]$ | Voto mayoritario                  |
| AdaBoost error    | $\varepsilon_t = \sum_i w_i \mathbf{1}[y_i \neq h_t(x_i)] / \sum_i w_i$        | $[0, 0.5]$ ; $< 0.5$ = útil       |
| AdaBoost $\alpha$ | $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ | Mayor $\alpha_t$ = más influencia |
| AdaBoost pesos    | $w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$                         | Aumenta pesos de errores          |
| AdaBoost pred.    | $\hat{y}(x) = \text{sign}(\sum_t \alpha_t h_t(x))$                             | Votación ponderada                |
| GB residuos       | $r_{im} = -\partial L(y_i, F(x_i)) / \partial F(x_i)$                          | Gradiente negativo                |
| GB (MSE)          | $r_{im} = y_i - F_{m-1}(x_i)$                                                  | Diferencia directa                |
| GB actualiz.      | $F_m(x) = F_{m-1}(x) + \eta h_m(x)$                                            | $\eta$ =lr, $h_m$ =árbol nuevo    |
| Sesgo-Varianza    | $E_{\text{total}} = \text{Sesgo}^2 + \text{Varianza} + \text{Ruido}$           | Tradeoff fundamental              |
| Bootstrap         | $P(\text{no sel.}) = (1 - 1/n)^n \approx e^{-1} \approx 0.368$                 | 63.2% únicos/muestra              |

Cuadro 9.1: Fórmulas esenciales del módulo

### 9.2 Guía Rápida de Implementación

Listing 9.1: Cheatsheet de código todos los modelos

```

1 # == ARBOL DE DECISION ==
2 from sklearn.tree import DecisionTreeClassifier
3 clf = DecisionTreeClassifier(
4     criterion='gini',          # 'gini' (default) o 'entropy'
5     max_depth=5,              # Pre-poda
6     min_samples_leaf=5,       # Pre-poda
7     ccp_alpha=0.01,           # Post-poda
8     random_state=1
9 )
10

```

```

11 # === RANDOM FOREST =====
12 from sklearn.ensemble import RandomForestClassifier
13 rf = RandomForestClassifier(
14     n_estimators=200,
15     max_features='sqrt',      # m = sqrt(p)
16     oob_score=True,
17     n_jobs=-1, random_state=1
18 )
19
20 # === ADABOOST =====
21 from sklearn.ensemble import AdaBoostClassifier
22 ada = AdaBoostClassifier(
23     n_estimators=100,
24     learning_rate=1.0,
25     random_state=1
26 )
27
28 # === GRADIENT BOOSTING =====
29 from sklearn.ensemble import GradientBoostingClassifier
30 gb = GradientBoostingClassifier(
31     n_estimators=100, learning_rate=0.1, max_depth=3,
32     subsample=0.8, n_iter_no_change=10,
33     validation_fraction=0.1, random_state=1
34 )
35
36 # === XGBOOST =====
37 from xgboost import XGBClassifier
38 xgb = XGBClassifier(
39     n_estimators=500, learning_rate=0.05, max_depth=6,
40     subsample=0.8, colsample_bytree=0.8,
41     gamma=1, reg_alpha=0.1, reg_lambda=1.0,
42     scale_pos_weight=3,      # Desbalance: neg/pos
43     tree_method='hist', enable_categorical=True,
44     early_stopping_rounds=10, random_state=1
45 )
46
47 # === LIGHTGBM =====
48 import lightgbm as lgb
49 lgbm = lgb.LGBMClassifier(
50     n_estimators=1000, learning_rate=0.05,
51     num_leaves=31,      # Control leaf-wise
52     min_child_samples=20, # Regularizacion
53     subsample=0.8, colsample_bytree=0.8,
54     reg_alpha=0.1, reg_lambda=1.0, verbose=-1
55 )
56 lgbm.fit(X_tr, y_tr, eval_set=[(X_val, y_val)],
57         callbacks=[lgb.early_stopping(50)])
58
59 # === CATBOOST =====
60 from catboost import CatBoostClassifier
61 cat = CatBoostClassifier(
62     iterations=1000, learning_rate=0.05,
63     depth=6, l2_leaf_reg=3, verbose=0
64 )
65 cat.fit(X_tr, y_tr, cat_features=['col1', 'col2'],
66        eval_set=(X_val, y_val), early_stopping_rounds=50)
67
68 # === BUSQUEDA DE HIPERPARAMETROS =====
69 from sklearn.model_selection import RandomizedSearchCV
70 from scipy.stats import loguniform, uniform, randint
71 dist = {'learning_rate': loguniform(0.01, 0.3),
72        'n_estimators': randint(50, 500)}
73 rs = RandomizedSearchCV(model, dist, n_iter=50,

```

```
74     scoring='f1_weighted', cv=5, n_jobs=-1, random_state=1)
75 rs.fit(X_train, y_train)
```



# Capítulo 10

## Guía de Estudio Definitiva

### 10.1 Prerrequisitos

| Prerrequisito                      | Nivel requerido   | Por qué es necesario                          |
|------------------------------------|-------------------|-----------------------------------------------|
| Python (pandas, numpy, matplotlib) | Intermedio        | Todos los notebooks lo usan                   |
| Estadística básica                 | Básico            | Probabilidad, media, varianza, distribuciones |
| Álgebra lineal                     | Básico            | Vectores y matrices para feature importances  |
| Teoría de la información           | Nociones          | Entropía y ganancia de información            |
| scikit-learn API                   | Básico-Intermedio | fit/predict/Pipeline/GridSearchCV             |
| Conceptos de ML                    | Básico            | Overfitting, cross-validation, métricas       |

### 10.2 Roadmap de Aprendizaje 4 Semanas

#### Semana 1 Árboles de Decisión

- Estudiar: capítulos 2 y 3 de esta guía (teoría CART, criterios de impureza)
- Implementar: árbol con Adult dataset; árbol de regresión con Auto-MPG
- Ejercicio: calcular manualmente entropía y Gini (ejercicios 1 y 2)
- Meta: entender CART, criterios de impureza, regularización

#### Semana 2 Bagging y Random Forest

- Estudiar: capítulo 4 de esta guía (bootstrap, OOB, decorrelación)
- Implementar: BaggingRegressor vs RandomForestRegressor
- Ejercicio: bootstrap manual y análisis de decorrelación (ejercicios 3 y 4)
- Meta: entender bootstrap, OOB,  $m$ -features, Random Forest

#### Semana 3 Boosting: AdaBoost y Gradient Boosting

- Estudiar: capítulos 5 y 6 de esta guía
- Implementar: AdaBoost y GradientBoosting con Pipeline completo
- Ejercicio: seguimiento manual de AdaBoost (ejercicio 5); curva de aprendizaje (ejercicio 6)
- Meta: entender residuos, learning rate, funciones de pérdida

### Semana 4 XGBoost, LightGBM y CatBoost

- Estudiar: capítulo 7 de esta guía + documentación oficial de cada librería
- Implementar: comparativa completa de los 3 en el mismo dataset
- Meta: saber cuándo usar cada uno, cómo configurar early stopping y tuning

### Consolidación Proyecto Integrador

- Aplicar pipeline completo: EDA  $\rightarrow$  preprocesamiento  $\rightarrow$  árbol  $\rightarrow$  RF  $\rightarrow$  boosting
- Comparar modelos con tabla de resultados (F1 train/test, tiempo, overfitting)
- Analizar importancias de features con todos los modelos
- Implementar SHAP values para el mejor modelo

## 10.3 Errores Comunes y Cómo Evitarlos

| Concepto           | Error común                         | Estrategia correcta                                                    |
|--------------------|-------------------------------------|------------------------------------------------------------------------|
| ccp_alpha          | Confundirlo con umbral de precisión | Visualizar árbol con distintos $\alpha$ ; observar simplificación      |
| Residuos en GB     | Creer que es $y - \hat{y}$ siempre  | Para MSE sí; para log-loss es el gradiente negativo de la pérdida      |
| lr vs n_estimators | Tratarlos como independientes       | Son complementarios; lr bajo requiere más árboles. Usar early stopping |
| OOB error          | Confundirlo con error en test       | OOB estima la generalización, no es exactamente igual a test           |
| Feature importance | Inferir causalidad                  | Solo indica relevancia predictiva. No causa-efecto                     |
| Leaf-wise (LGBM)   | Ignorar riesgo de overfitting       | Controlar con num_leaves, no solo max_depth                            |
| Target encoding    | Aplicar sin protección              | Siempre usar TargetEncoder de sklearn o CatBoost                       |

## 10.4 Tabla Resumen Final

| Necesidad                        | Algoritmo                    | Parámetro clave               |
|----------------------------------|------------------------------|-------------------------------|
| Interpretabilidad total + reglas | Árbol podado (depth=3-5)     | ccp_alpha / max_depth         |
| Baseline rápido y robusto        | Random Forest                | max_features='sqrt'           |
| Precisión, datos medianos        | Gradient Boosting / AdaBoost | lr + n_estimators             |
| Mejor precisión general          | XGBoost                      | gamma, reg_alpha/lambda       |
| Datos muy grandes (>500k)        | LightGBM                     | num_leaves, min_child_samples |
| Muchas variables categóricas     | CatBoost                     | cat_features, l2_leaf_reg     |
| Máxima precisión (competencia)   | Stacking RF+XGB+LGBM+CAT     | Meta-modelo logístico         |

# Capítulo 11

## Conclusiones y Próximos Pasos

### 11.1 Aprendizajes Principales

- ✓ **Los árboles siempre sobreajustan sin regularización.** Sin podado: train=100 %, test=77 %. La regularización es obligatoria.
- ✓ **Bagging reduce varianza; Boosting reduce sesgo.** Son filosofías complementarias. Si el árbol base es estable pero sesgado, Boosting es más útil. Si es inestable, Bagging primero.
- ✓ **Random Forest: robusto con poco tuning.** Solo ajustando `n_estimators` y `max_depth` ya da buenos resultados. Es el algoritmo “seguro” cuando no se sabe qué usar.
- ✓ **XGBoost/LightGBM/CatBoost son el estado del arte para tabular.** En prácticamente cualquier competencia con datos tabulares, uno de estos tres gana. En producción, LightGBM es favorito por velocidad.
- ✓ **El escalado NO ayuda a los árboles.** Invariantes al escalado por definición. No desperdiciar tiempo en `StandardScaler`.
- ✓ **Las categóricas requieren codificación (excepto CatBoost).** AdaBoost y GB clásico no soportan categóricas. XGBoost y LightGBM sí con configuración.
- ✓ **Los valores nulos no son problema (excepto AdaBoost).** XGBoost, LightGBM y CatBoost (en numéricas) manejan NaN nativamente.

### 11.2 Próximos Pasos Recomendados

- Implementar **SHAP values** para interpretabilidad post-hoc de ensambles (más informativo que `feature_importances_`).
- Explorar **Optuna** para búsqueda bayesiana de hiperparámetros (más eficiente que `RandomizedSearchCV`).
- Implementar **stacking**: árbol + RF + XGBoost + LGBM como base; meta-modelo logístico.
- Explorar **calibración de probabilidades** (Platt scaling, isotonic regression) para modelos de boosting.
- Aplicar **curvas de aprendizaje** para diagnosticar sesgo vs varianza antes de elegir el algoritmo.
- Investigar **pérdidas personalizadas** (focal loss para clases muy desbalanceadas).

*Guía Maestra generada el 21 de mayo de 2026  
Emanuel Cabrera Novoa ù Análisis Predictivo de Datos ù Docente: Jhon Quiza*